Big Data(BCS061/BCDS-601/KOE-097

Unit – 4 Hadoop Ecosystem & YARN, NOSQL,MONGODB,SPARK,SCALA

IV	**Hadoop Eco System and YARN**: Hadoop ecosystem components, schedulers, fair and capacity, Hadoop 2.0 New Features - NameNode high availability, HDFS federation,MRv2, YARN, Running MRv1 in YARN. **NoSQL Databases:** Introduction to NoSQL **MongoDB:** Introduction, data types, creating, updating and deleing documents, querying, introduction to indexing, capped collections **Spark:** Installing spark, spark applications, jobs, stages and tasks, Resilient Distributed Databases, anatomy of a Spark job run, Spark on YARN **SCALA:** Introduction, classes and objects, basic types and operators, built-in control structures, functions and closures, inheritance.

Edushine Classes

Download Notes : https://rzp.io/rzp/vLNI7dA

EduShine CLASSES
By- ARMAN ALI

❖ **Hadoop Ecosystem & Its Component : Already Discussed in Unit 2**

❖ **Fair Scheduler & Capacity Scheduler : Already Discussed in Unit 2**

📝 Hadoop 2.0 New Feature: NameNode High Availability

In **Hadoop 1.0**, there was only **one NameNode** (master that manages file storage).

☞ If that NameNode failed, the whole system **stopped working**. This was a big problem!

◆ What is NameNode High Availability?

In **Hadoop 2.0**, there are **two NameNodes**:

i. **Active NameNode** – Handles all file operations (read/write)

ii. **Standby NameNode** – Just waits quietly and watches

☞ If the Active one fails, the Standby **automatically takes over**, so **Hadoop keeps running** without stopping.

◆ How it works :

- Both NameNodes share the **same file system data**
- They use a system called **"JournalNodes"** to keep track of every change
- If Active fails, the Standby uses the journal and becomes Active

EduShine CLASSES By- ARMAN ALI

📝 Hadoop Federation :

Hadoop Federation is a feature added in **Hadoop 2.0** to fix a big problem in HDFS.

◆ The Problem (Before Federation)

In old HDFS, there was **only one NameNode**

It managed **all the files** in the system

👉 So, it could become **overloaded** and **slow**

◆ What is Federation?

Federation means using multiple NameNodes in one Hadoop cluster.

Each NameNode manages **its own part** of the file system (called a **namespace**)

All NameNodes work **independently** but **share the same DataNodes**

◆ How it Works (Very Simple)

- **More than one NameNode**
- Each one has its own **namespace** (like a separate folder system)
- All NameNodes **use the same storage** (DataNodes)

👉 Think of it like **multiple managers** managing different departments in one office.

RRSIMT CLASSES

EduShine CLASSES
By- ARMAN ALI

📝 Short Note on MRv2 (MapReduce Version 2) –

MRv2 is the new version of **MapReduce** introduced in **Hadoop 2.0**.
It is also called **YARN-based MapReduce**.

◆ What is MRv2?

In **MRv1**, the JobTracker did **everything** – it managed jobs, resources, and tasks.
☞ This caused slowdowns and **one-point failure**.
MRv2 solves this by **separating job management and resource management** using **YARN**.

◆ Key Parts of MRv2:

i. **ResourceManager (RM)** – Gives resources to run jobs
ii. **NodeManager (NM)** – Runs on each machine, manages containers (where tasks run)
iii. **ApplicationMaster (AM)** – Handles each job separately

◆ Benefits of MRv2:

- Better **scalability** (can handle more jobs)
- Supports **other applications** (not just MapReduce)
- More **efficient and flexible**
- Removes **bottlenecks** of old JobTracker

EduShine CLASSES
By- ARMAN ALI

📝 How to Run MRv1 in YARN :

Even though **YARN** was introduced in Hadoop 2.0 to replace the old MapReduce system (**MRv1**), you can **still run old MRv1 programs** on YARN with **very little change**.

◆ Step-by-Step (Easy Explanation):

1.Write your MRv1 job – This is the same MapReduce code you wrote in Hadoop 1.x (using JobConf, Mapper, Reducer, etc.)

2.Submit the job to YARN – In Hadoop 2.x or 3.x, when you submit your MRv1 job, YARN will **automatically run it** using the **MapReduce framework on YARN** (also called **MapReduce v2** runtime).

3.No need to change code – You don't have to rewrite your old MRv1 jobs.

👉 YARN understands and supports them.

4.YARN takes care of execution –

Old MRv1 Code → Submit to YARN → Runs using MRv2 framework

- **ResourceManager** gives resources
- **NodeManagers** run your map/reduce tasks in containers
- An **ApplicationMaster** is started for your job

Big Data(BCS061/BCDS-601

EduShine CLASSES
By- ARMAN ALI

📝 What is NoSQL Database?

NoSQL means "**Not Only SQL**".

It is a type of database used to **store and manage large amounts of unstructured, semi-structured, or structured data**.

👉 NoSQL is flexible, fast, and handles **Big Data** better than traditional SQL databases.

✅ Advantage	👆 Simple Meaning
Flexible Data Model	You can store any kind of data (text, images, etc.)
High Speed	Works faster for large amounts of data
Easy to Scale	Can handle more data by adding more servers
Handles Big Data	Great for social media, IoT, logs, etc.
No Complex Joins	Simpler queries, better performance

✕ Disadvantage	⚠ Simple Meaning
No Standard Query Language	Different NoSQL databases use different syntax
Less Support for Complex Queries	Not ideal for reporting or joins
Less Mature	Not as old or tested as SQL databases
Data Consistency Issues	Focus more on speed, may lose consistency

? Type	Example	What It Stores
1. **Document**	MongoDB	Stores data like JSON (documents) – flexible format
2. **Key-Value**	Redis	Stores data as key → value (like a dictionary)
3. **Column-based**	Cassandra	Stores data in columns instead of rows – good for big tables
4. **Graph**	Neo4j	Stores data as nodes and relationships – used for social networks, maps

EduShine CLASSES
By- ARMAN ALI

📝 What is MongoDB?

MongoDB is a popular **NoSQL database** that stores data in a **document format** (like JSON).

👉 Instead of tables and rows like SQL, MongoDB uses **collections and documents**.

📁 Example:

A document in MongoDB looks like this:

```
json
{
"name": "Arman",
"age": 21, "city": "Delhi"
}
```

Big Data(BCS061/BCDS-601

EduShine CLASSES
By- ARMAN ALI

✅ Features of MongoDB :

🔷 Feature	👉 Simple Meaning
📄 Document-Oriented	Stores data in flexible JSON-like format
⚡ High Performance	Very fast to read/write data
🔄 Easy to Scale	Can add more servers when data grows
🔍 Rich Query Language	You can filter, sort, and search data easily
🎁 Schema-less	Each document can have different structure
🔧 Indexing	Faster searching using indexes
🔗 Aggregation	Can group and process data like in reports
🌐 Supports many languages	Works with Java, Python, Node.js, etc.

Big Data(BCS061/BCDS-601

EduShine CLASSES
By- ARMAN ALI

❖ Data Types Used in MongoDB :

Data Type	Meaning / Example
String	Text → "name": "Arman"
Number	Integer or Float → "age": 21
Boolean	True or False → "passed": true
Date	Date and Time → "dob": ISODate("2000-01-01")
Array	List of values → "skills": ["C", "Java"]
Object	Nested document → "address": {"city": "Delhi"}
Null	Empty value → "middleName": null
ObjectId	Special ID for each document (_id)
Binary Data	For storing files like images or PDFs

EduShine CLASSES
By- ARMAN ALI

📝 How to Update and Delete Documents in MongoDB :

MongoDB stores data in **documents** (like JSON). You can **update** or **delete** these documents using simple commands.

✅ 1. Update a Document

🛠 Syntax:

```
db.collection.updateOne(filter, update)
```

🔍 Example:

```
db.students.updateOne(
  { name: "Arman" },          // 🔍 Find document where name is Arman
  { $set: { age: 22 } }       // ✏️ Set age to 22
)
```

2. Update Many Document

```
db.students.updateMany(
  { passed: false },
  { $set: { passed: true } }
```

✅ Updates all documents where passed is false.

EduShine CLASSES
By- ARMAN ALI

❌ 3. Delete a Document:

🛠 Syntax:

db.collection.deleteOne(filter)

Example -1 :

db.students.deleteOne({ name: "Arman" })

✅ Deletes **one** document where name is Arman.

Example – 2 :

db.students.deleteMany({ age: { $lt: 18 } })

✅Deletes **all** documents where age is less than 18.

Operation	Command	Meaning
Update One	updateOne()	Change one matching document
Update Many	updateMany()	Change all matching documents
Delete One	deleteOne()	Remove one matching document
Delete Many	deleteMany()	Remove all matching documents

EduShine CLASSES
By- ARMAN ALI

🔍 What is a Query in MongoDB?

A **query** is a way to **search or find data** in MongoDB.

☞ Just like searching in Google, you can search for documents in a **collection** using queries.

Example : Find Students Name “Rahul”

db.students.find({ name: “Rahul" })

Example : Queries with Conditions

db.students.find({ age: { $gt: 18 } })

Operator	Meaning	Example
$gt	Greater than	{ age: { $gt: 18 } }
$lt	Less than	{ age: { $lt: 18 } }
$eq	Equal to (default)	{ name: "Arman" }
$ne	Not equal	{ age: { $ne: 20 } }
$in	In a list	{ city: { $in: ["Delhi", "Noida"] } }

EduShine CLASSES
By- ARMAN ALI

⚙What is Indexing in MongoDB?

An **index** in MongoDB is like a **shortcut** to find data **faster**.

👉 Without indexes, MongoDB **searches every document** (slow).

👉 With indexes, MongoDB can **quickly jump** to the correct data (fast).

Example :

```
db.students.createIndex({ name: 1 })
```

👍 Advantage	💬 Simple Meaning
Faster queries	Search is much quicker
Saves time	Especially useful with big data
Supports sorting	Helps sort data faster

EduShine CLASSES
By- ARMAN ALI

📝 What is a Capped Collection in MongoDB?

A **Capped Collection** is a special type of collection in MongoDB that:

- Has a **fixed size**
- Works like a **circular queue**
- **Automatically removes old data** when it's full

🔷 Feature	👍 Easy Meaning
Fixed size	You set max size (in bytes or document count)
Auto overwrite	Oldest data is deleted when full
Fast write	Very fast insert operations
Insert-only	No delete or update of size-changing fields
Keeps order	Maintains the order of insertion

EduShine CLASSES
By- ARMAN ALI

What is Apache Spark?

Apache Spark is a fast and powerful tool used to **process and analyze big data**.

- It works on a large amount of data quickly
- It can do many tasks like data processing, machine learning, and streaming
- Spark is better and faster than older tools like Hadoop MapReduce

Feature	Simple Meaning
Fast Processing	Very fast in handling big data
Easy to Use	Has simple programming languages like Python, Java, Scala
Works in Memory	Processes data in RAM (fast memory)
Supports Multiple Tasks	Can do batch jobs, streaming, machine learning, and graph processing
Scalable	Can work on small or very big data
Runs Everywhere	Works on your laptop or big clusters

EduShine CLASSES
By- ARMAN ALI

🛠 How to Install Spark :

Step 1: Install Java

- Spark needs Java to run
- Make sure Java is installed on your computer
- Check by running in terminal/cmd:

java -version

Step 2: Download Spark

- Go to Apache Spark website: https://spark.apache.org/downloads.html
- Choose a Spark version and package type (choose pre-built for Hadoop)
- Download the file and unzip it to a folder

Step 3: Set Environment Variables (Optional but recommended)

- Add Spark's bin folder path to your system’s PATH
- This lets you run Spark commands from any terminal

EduShine CLASSES
By- ARMAN ALI

Step 4: Run Spark

- Open terminal or command prompt
- Go to Spark folder's bin directory
- Run Spark shell (interactive mode):

📝 Spark Application, Jobs, Stages, and Tasks:

1. What is a Spark Application?

A **Spark Application** is a program you write using Spark to process data.
It can do things like read data, transform it, and save results.
For example, a program that counts words in documents is a Spark application.

2. What is a Job?

- When you run an action in Spark (like count(), collect()), Spark creates a **job**.
- A **job** is a big task that Spark must do to get the final result.

EduShine CLASSES
By- ARMAN ALI

3. What is a Stage?

Spark breaks each job into smaller parts called **stages**.
Each stage contains tasks that can be run in parallel (at the same time).
Stages are separated by **shuffle operations** (when data needs to be moved around).

4. What is a Task?

A **task** is the smallest unit of work in Spark.
Each task works on a small piece of data (called a partition).
Many tasks run in parallel to finish a stage faster.

Simple Example to Understand

- You write a Spark program → Spark Application
- You call count() on data → Spark starts a Job
- Job breaks into 2 Stages → Stage 1 and Stage 2
- Each Stage runs many Tasks → each task processes part of data

EduShine CLASSES
By- ARMAN ALI

What is Resilient Distributed Dataset (RDD)?

RDD is a special way Spark stores data to make big data processing easy and safe.

Term	Easy Meaning
Resilient	If part of data is lost, Spark can recover it easily without restarting everything.
Distributed	Data is split and stored across many computers (nodes).
Dataset	It's a collection or group of data.

How RDD Works:

- Spark breaks your big data into small pieces and keeps them on different computers.
- If a computer crashes, Spark can rebuild the lost data from other pieces (using something called lineage).
- This makes data processing **fast** and **fault-tolerant** (safe from errors).

EduShine CLASSES
By- ARMAN ALI

Anatomy of a Spark Job Run :

When you run a job in Spark, it goes through these simple steps:

1. **User Writes Code**
   You write a Spark program (like reading data, transforming it, and asking for results).
2. **Job Created**
   When you ask Spark to do something (like count()), Spark creates a **job**.
3. **Job Divided into Stages**
   Spark splits the job into smaller **stages** based on where data needs to be shuffled or moved.
4. **Stages Divided into Tasks**
   Each stage is broken into many **tasks** that work on small parts of data (called partitions).
5. **Tasks Sent to Executors**
   These tasks are sent to different computers (called executors) in the cluster to run in parallel.

EduShine CLASSES
By- ARMAN ALI

6. Tasks Run and Results Collected

Executors run the tasks, process data, and send results back.

7. Job Completes

After all tasks finish, Spark gives you the final result.

📝 What is Running Spark on YARN?

1. What is YARN?

- **YARN** stands for **Yet Another Resource Negotiator**
- It is like a **manager** that controls and shares resources (CPU, memory) in a big cluster of computers
- Helps many applications (including Spark) run together smoothly on a big cluster

2. Why run Spark on YARN?

- YARN **manages resources** so Spark doesn't use too much or too little
- Allows Spark to **run on big clusters** with many computers
- Lets Spark **share resources** with other applications like Hadoop MapReduce

EduShine CLASSES
By- ARMAN ALI

3. How does Spark run on YARN?

- When you start a Spark job, Spark asks YARN for resources (like computers and memory)
- YARN allocates resources and starts Spark's **executors** (workers) on different computers
- Spark runs tasks on these executors to process data in parallel
- YARN keeps track of resource use and job progress

👍 Advantage	Simple Meaning
Resource management	YARN controls memory and CPU usage
Scalability	Can run Spark on many computers
Multi-tenancy	Run Spark with other apps together
Fault tolerance	YARN can restart Spark tasks if they fail

EduShine CLASSES
By- ARMAN ALI

What is Scala ?

Scala is a programming language that is:

- Simple and powerful
- Works well with big data tools like Spark
- Combines features of **Java** and **functional programming**
- Helps you write less code but do more things

1. What is a Class in Scala?

- A **class** is like a **blueprint** or **template** to create objects (things).
- It defines **fields** (properties) and **methods** (actions) that the object can have.

2. What are Fields in Scala?

- **Fields** are variables inside a class that hold data or information about the object.
- For example, a Car class can have fields like color, model, speed.

EduShine CLASSES
By- ARMAN ALI

3. What are Methods in Scala?

- **Methods** are functions inside a class that describe what the object can do.
- For example, a Car class can have methods like start(), stop(), accelerate().

Example :

```
class Car {
  // Fields
  var color: String = "Red"
  var speed: Int = 0
  // Method
  def start(): Unit = {
    println("Car started")
  }
  def accelerate(amount: Int): Unit = {
    speed += amount
    println(s"Car speed increased to $speed")
  }
}
```

EduShine CLASSES
By- ARMAN ALI

📝 What is a Singleton Object in Scala?

A **singleton object** in Scala means:

👉 **Only one object** of that type exists in the whole program.

◆ Why is it called "Singleton"?

"Single" means **one**.

So **Singleton Object** = **Only one object** of that class is created.

◆ How do you create a Singleton Object?

In Scala, you use the keyword **object** instead of **class**.

```
object MySingleton {
  def sayHello(): Unit = {
    println("Hello from Singleton!")
  }
}
```

EduShine CLASSES
By- ARMAN ALI

✅ 1. Basic Types in Scala

Scala has many built-in data types to store different kinds of values:

Type	Example	Meaning (in simple words)
Int	5, 100	Whole numbers
Double	5.5, 3.14	Decimal numbers
Boolean	true, false	Yes/No values
Char	'A', 'z'	Single characters
String	"Hello"	Group of characters (text)
Long	1234567890L	Large whole numbers
Float	3.14f	Decimal number with f at the end

Big Data(BCS061/BCDS-601

EduShine CLASSES
By- ARMAN ALI

✅ 2. Operators in Scala

◆ Arithmetic Operators (for numbers):

Type	Operator	Example	Meaning (Easy Words)	Result Example
Arithmetic	+	5 + 3	Add two numbers	8
	-	10 - 4	Subtract numbers	6
	*	6 * 2	Multiply	12
	/	8 / 2	Divide	4
	%	7 % 3	Remainder after division	1
Comparison	==	a == b	Equal to	true / false
	!=	a != b	Not equal to	true / false
	>	a > b	Greater than	true / false
	<	a < b	Less than	true / false
	>=	a >= b	Greater than or equal	true / false
	<=	a <= b	Less than or equal	true / false
Logical	&&	true && false	AND (both must be true)	false
	`		`	`true
	!	!true	NOT (opposite of value)	false

EduShine CLASSES
By- ARMAN ALI

📝 Built-in Control Structures in Scala

Control structures help you **control the flow** of your program — like making decisions, repeating tasks, or choosing between options.

Here are the **main built-in control structures in Scala** explained simply:

✅ 1. if / else – Decision Making

Used to choose between two or more options.

Example:

```
val age = 18
if (age >= 18) {
  println("You are an adult")
} else {
  println("You are a minor")
}
```

✅ 2. if / else if / else – Multiple Choices :

Example :

```
val marks = 85
if (marks >= 90) {
  println("Grade A")
} else if (marks >= 75) {
  println("Grade B")
} else {
  println("Grade C")
}
```

EduShine CLASSES
By- ARMAN ALI

✅ 3. for loop – Repeat for a number of times

Used to repeat a block of code.

Example :

```
for (i <- 1 to 5) {
  println(i)
}
```

✅ 4. while loop – Repeat while condition is true

```
var i = 1
while (i <= 3) {
  println("Hello")
  i += 1
}
```

✅ 5. do...while loop – Do first, then check condition :

```
var i = 1
do {
  println("Scala")
  i += 1
} while (i <= 2)
```

EduShine CLASSES
By- ARMAN ALI

✅ Various Functions Used in Scala :

In Scala, **functions** are blocks of code that do some work. You can **reuse** them to avoid writing the same code again and again.

Type	Example	Easy Meaning
Simple Function	def greet(): Unit = println("Hi!")	No input, just prints something
With Parameters	def square(x: Int): Int = x * x	Takes input, returns result
Recursive Function	def fact(n: Int): Int = if (n==0) 1 else n * fact(n-1)	Calls itself
Anonymous Function	val add = (a: Int, b: Int) => a + b	Function without a name
Higher-order Function	def operate(x: Int, f: Int => Int): Int = f(x)	Takes another function as input

EduShine CLASSES
By- ARMAN ALI

✅ What is a Closure in Scala?

A **closure** is a **function that remembers the values of variables** from where it was created.

◆ Example:

scala

```scala
val factor = 3
val multiply = (x: Int) => x * factor
```

- `multiply` is a **closure**.
- It uses the `factor` variable from **outside the function**.

Even if you use `multiply` somewhere else, it remembers `factor = 3`.

◆ Easy Meaning:

👉 A **closure** = function + the environment it remembers (like saved variables).

EduShine CLASSES
By- ARMAN ALI

✅ What is Inheritance in Scala?

Inheritance means one class **gets features** (fields + methods) of another class.

Example :

```
class Animal {
  def sound(): Unit = {
    println("Some sound")
  }
}
class Dog extends Animal {
  def bark(): Unit = {
    println("Woof!")
  }
}
```

Download Notes : https://rzp.io/rzp/vLNI7dA

EduShine CLASSES
By- ARMAN ALI

Thank You….

Download Notes : https://rzp.io/rzp/vLNI7dA